
Leaky RND: Regenerating Curiosity for Exploration in Reward-Sparse Environments

Chris He Yuntian Zhao Zheyu Chen Huei-Guo Chang Chia-Han Chen
University of California, San Diego
{xih078, yuz299, zhc075, huc028, chc016}@ucsd.edu

Abstract

In environments where reward is sparse, intrinsic signals such as curiosity (i.e., explore unexplored states) are often used to guide an agent’s exploration. However, in environments with small state spaces, such as ARC-AGI-3, curiosity can saturate quickly, before the agent discovers the specific sequence of actions needed to obtain a reward. In this work, we present Leaky RND, a curiosity-based exploration method that continuously regenerates curiosity for states that have not been visited recently. Paired with a state representation learned through inverse-dynamics modeling, Leaky RND reduces the average number of steps to complete a level by 35% over the best baseline. Nonetheless, it solves no levels beyond those previous methods already solve, highlighting a fundamental limitation of curiosity-based methods: they cannot effectively exploit the information they collected through exploration.

1 Introduction

Curiosity drives how humans interact with the world. It leads people toward places and things they are less familiar with, thus helping them gain a better understanding of the world. In short, curiosity directs our exploration when external incentives don’t exist. Hence, the notion of curiosity, or novelty, is introduced [10, 15] to address the question of how to explore in an environment with sparse rewards.

Previous work demonstrated the effectiveness of curiosity as an intrinsic source of reward, yielding state-of-the-art performances on notoriously hard games like Montezuma’s Revenge [5, 12]. In addition, unlike hand-designed intermediate rewards, curiosity-as-a-reward does not require knowledge about the environment, making it a general exploration strategy usable in arbitrary environments.

Although these approaches demonstrated more efficient exploration in environments with large state space, they underperform in small environments with extremely-sparse rewards like ARC-AGI-3 [6], where a very specific sequence of actions is needed to earn a non-zero reward. In other words, being curious doesn’t necessarily lead to reward. An agent can explore the environment thoroughly — to the point that every state is familiar and curiosity saturates (approaches zero) [17] — without stumbling upon the reward. Then, without curiosity signal, the agent reverts back to random exploration.

In this work, we introduced Leaky RND, which prevents curiosity saturation by gradually leaking information about which states have been explored, thereby renewing the agent’s curiosity toward states it has not visited recently. This reduces the average number of steps required to complete a level by 35%. Nonetheless, we argue that the fundamental problem remains open: curiosity and reward are not perfectly aligned. Reward sometimes requires systematic trial-and-error to reproduce a specific action sequence, whereas curiosity supplies only an undirected exploration of the state space.

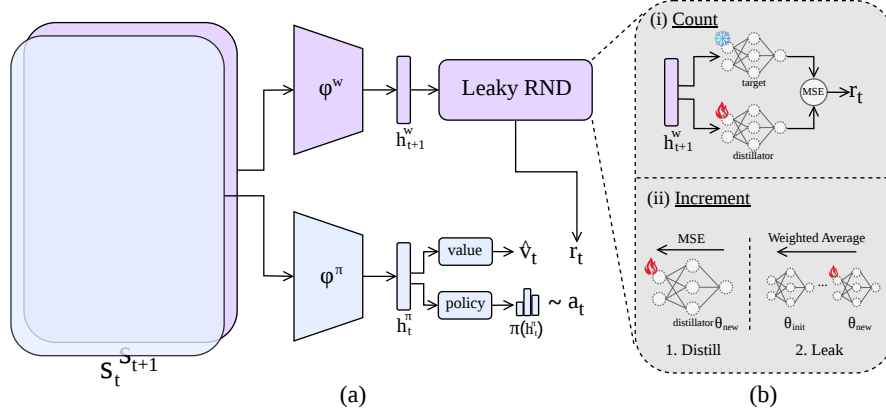


Figure 1: System overview. The agent (blue) observes the current state s_t and chooses an action a_t . The harness (purple) steps the environment, counts (b)(i) how many times s_{t+1} is visited, produces a reward r_t for the agent, and increments s_{t+1} 's visit count.

2 Background

2.1 Exploration in Reward-Sparse Environments

Reinforcement Learning (RL) methods allow agents to learn from interacting with the environment, using environment feedback to improve their behavior. However, obtaining a reward in the first place can be a non-trivial task for a randomly initialized agent. Human demonstration data [7, 14] can be used to warm-start the agent. Yet, we ultimately envision agents capable of learning on their own, in any environment, without human assistance.

Current approaches mainly drive exploration through intrinsic rewards that favor novel or unexplored states. They differ primarily in how they quantify the degree to which a state has been explored, and can be grouped into two families: count-based and prediction-based methods.

Count-based methods. Count-based methods [3, 5, 11] track how many times each state is visited, treating more visits as better explored. A core challenge is state representation — it's undesirable to overcount distinct observations (e.g., game screenshots) that map to the same underlying state. Pseudo-count [3] addresses this via a density model that generalizes counts across similar states, while RND [5] leverages interpolation between similar states when distilling a random network.

Prediction-based methods. Prediction-based methods [2, 4, 12] utilize tasks like predicting the state transition, assuming better task performance at a state means more familiarity with that state. Recent works aim to tackle the noisy-TV problem [4], where agents reward hack by fixating on states with stochastic transitions — hence prediction error never decreases. ICM [12] introduced an inverse dynamic model so that prediction is made in a state space unaffected by uncontrollable noise, while Disagreement [13] instead trained multiple forward dynamic models, measuring the variance between their predictions rather than the prediction error itself.

2.2 ARC-AGI-3

ARC-AGI-3 [6] is a benchmark that evaluates an agent's ability to explore and plan in unseen environments. Each environment is a 64×64 grid-world puzzle composed of multiple levels of increasing difficulty. The only feedback an agent receives is whether it has completed a level, making the reward extremely sparse. Compounding this, solving a level often requires executing a specific sequence of actions exactly, so the probability that a random agent stumbles onto a solution decays exponentially with the length of that sequence. Frontier models like GPT-5.5 and Opus 4.7 reportedly score only 0.43% and 0.18% [1], confirming ARC-AGI-3 as a demanding testbed for exploration under sparse reward.

3 Methodology

Our system can be viewed as an agent with a harness. As shown in Figure 1, the agent (blue) observes the state s_t and chooses an action a_t . The harness (purple) steps the environment, and produces a reward r_t for the agent.

3.1 Harness

The harness consists of two modules: a world encoder ϕ^w that maps the game screenshot (raw pixels) s_{t+1} to a hidden representation h_{t+1}^w ; and Leaky RND, which approximates the number of times h_{t+1}^w is visited with a distillation loss $\mathcal{L}_{\text{Distill}}$.

3.1.1 World Encoder

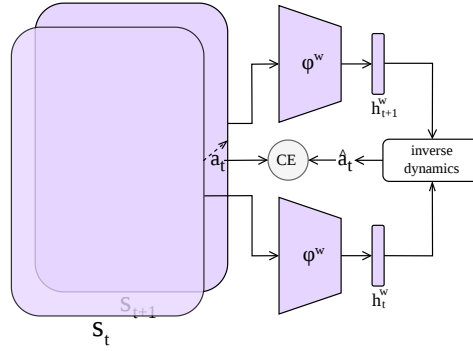


Figure 2: World encoder trained with an inverse-dynamics objective.

Issue of raw pixels. We ultimately wish to count how many times each distinct state has been visited. Counting directly in pixel space is undesirable: two frames can carry the same task-relevant information yet differ in pixels (e.g., the same agent position but a shorter energy bar), causing us to treat one state as many and overcount. We therefore want a representation that discards such task-irrelevant variation.

Learning representation with inverse-dynamics. To obtain this representation, we train a world encoder ϕ^w with an inverse-dynamics objective (Figure 2). For each transition (s_t, a_t, s_{t+1}) , we encode both states and ask an inverse-dynamics head IDM to predict the action $\hat{a} = \text{IDM}(\phi^w(s_t), \phi^w(s_{t+1}))$ that produced the transition, optimized with

$$\mathcal{L}_{\text{IDM}} = \text{CrossEntropy}(\hat{a}_t; a_t) \quad (1)$$

Such formulation forces ϕ^w to keep only what the agent can control and ignore the rest, so frames that differ only in task-irrelevant pixels collapse to the same representation and are no longer overcounted.

Online training. ϕ^w is trained alongside the agent, improving as the agent explores more states. Due to this, the same state s_t can be mapped to different representations h_t^w as training progresses, effectively resetting the count of s_t . To address this, we warm-start ϕ^w with 20 episodes of data collected with a random policy, and treat subsequent encoder drifts (empirically small) as a natural source of leakage. Following Burda et al. [4], we also tested a random encoder frozen at start, and presented the results in Section 4.2.

3.1.2 Leaky RND

Leaky RND is used to keep track of how many times each h_t^w is visited, and generate reward for the agent. Leaky RND is comprised of two modules: a randomly initialized distillation target f_ψ and a distillator $\hat{f}_\theta$.

Counting number of visits. f_ψ assigns each state h_t^w a random vector $f_\psi(h_t^w)$. Each time the agent visits a state h_t^w , the distillator $\hat{f}_\theta$ is allowed to distill f_ψ on that state (Figure 1(b)(ii)) via

$$\mathcal{L}_{\text{Distill}} = \text{MSE}(\hat{f}_\theta(h_t^w); f_\psi(h_t^w)) \quad (2)$$

The more the agent visits a state h_t^w , the better $\hat{f}_\theta$ is at distilling f_ψ at that state. Hence, distillation error can be used as a curiosity signal (Figure 1(b)(i)): an action a_t leading to a less-visited state h_{t+1}^w receives a higher reward $r_t = \mathcal{L}_{\text{Distill}}(h_{t+1}^w)$ due to the higher distillation error. In practice, because $|\mathcal{L}_{\text{Distill}}|$ can vary greatly as more distillation is done, we normalize it using running statistics.

Curiosity saturation. A key limitation of vanilla RND is that distillation error tends to saturate toward zero as states are visited repeatedly, eliminating the curiosity signal entirely. This is especially severe in small or structured state spaces like ARC-AGI-3, where the agent exhausts novel states quickly. Standard mitigations such as reducing $\hat{f}_\theta$ ’s capacity or lowering its learning rate η can delay saturation, but do not fundamentally resolve it — the signal eventually collapses regardless.

Leak mechanism. To continuously regenerate curiosity, we introduce a leak mechanism that periodically pulls the distillator’s parameters back toward their random initialization. After each distillation step, we update (Figure 1(b)(ii)):

$$\theta_{\text{new}} = (1 - \mu) \cdot \theta_{\text{new}} + \mu \cdot \theta_{\text{init}} \quad (3)$$

where $\mu = 0.05$ controls the degree of leak. This formulation acts as a soft reset — gradually increasing distillation error for states that have not been recently visited, thereby restoring their curiosity signal. The intuition is that revisiting previously explored states under a new policy may yield genuinely novel behavior.

3.2 Agent

The agent consists of a Convolutional Neural Network (CNN) [9] backbone ϕ^π and a policy and a value head, trained via Proximal Policy Optimization (PPO) [16]. Detailed training setup is presented in Appendix A.

Intrinsic reward. The agent is trained purely on the intrinsic reward r_t supplied by the harness. Training is stopped when the agent receives its first extrinsic reward (i.e., completes a level). We experimented with transferring the learned ϕ^w to the next level. However, we observed it leads to minimal gains, because even though feature detectors can be reused, ϕ^w is unable to learn the (more conceptual) game mechanisms (e.g., being in the way of an orange blob will be eaten) shared between levels.

Agent-specific encoder. We trained separate encoders ϕ^π and ϕ^w for the agent and the harness, since they need to perform fundamentally different tasks. We observed that a shared encoder leads to degraded performance due to gradient interference between $\mathcal{L}_{\text{IDM}}$ and $\mathcal{L}_{\text{PPO}}$.

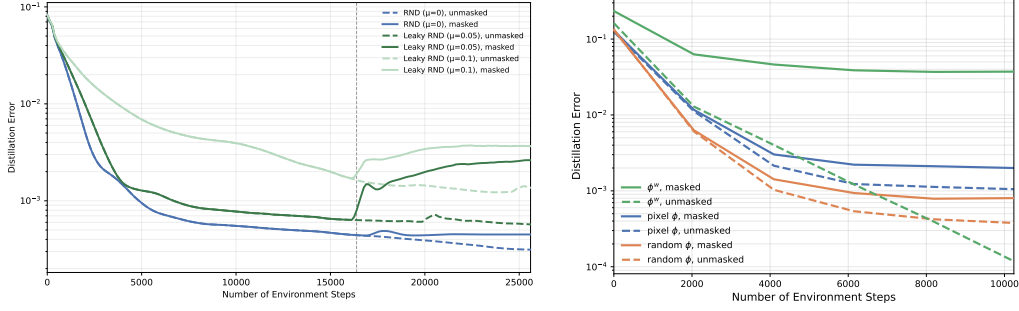
4 Results

4.1 Leaky RND Regenerates Curiosity

To study the behavior of Leaky RND, we monitored the distillation error of a single frequently-visited state near the agent’s spawn in 1s20 level 2, using a frozen pre-trained world encoder ϕ^w to eliminate confounds. Figure 3a shows that as the agent repeatedly visits this state, its distillation error dropped and plateaued, and a higher μ saturates slower.

At $\sim 16,000$ steps (128 updates), we masked actions that can reach the monitored state, so that it is no longer visited (solid lines). For vanilla RND (blue), the curiosity for the state stays low, regardless of whether it’s visited (dashed) or not (solid). In contrast, with Leaky RND (green), curiosity for the state quickly regenerates as the state becomes unvisited (green, solid).

Nonetheless, the regeneration is smaller (i.e., less differentiation between visited and recently-unvisited states) for a larger μ (light green), due to a higher curiosity floor (light green, dashed). Hence, we used $\mu = 0.05$ for all subsequent experiments.



(a) Distillation error of a single monitored state under RND and Leaky RND, before and after it is masked from further visits. (b) Distillation error under pixel ϕ , random ϕ , and ϕ^w for a masked, never-visited state (solid) and a normally-visited state (dashed).

Figure 3: Probing the curiosity signal of Leaky RND on 1s20 level 2.

4.2 Counting in Representation Space

The reliability of count-based exploration hinges on the space in which states are counted: if distinct states map to nearby representations, novelty leaks between them and the count becomes meaningless. To test this, we ran Leaky RND on three encoders — pixel ϕ (raw pixels), random ϕ (a randomly initialized CNN, following [4]), and ϕ^w (warm-started) — tracking the distillation error of a masked, never-visited state (solid) against a normally-visited one (dashed) (Figure 3b).

Under pixel ϕ (blue) and random ϕ (orange), the masked state’s error falls almost as fast as the visited state’s: both encoders map visually similar but distinct states to nearby representations, allowing the distillator to interpolate from visited states to unvisited ones. Counting in ϕ^w removes this leakage. Because inverse-dynamics training forces distinct states apart, the masked state retains a high error throughout (green, solid) while the visited state’s decays (green, dashed) — a persistent gap that distinguishes unvisited from explored states, and the prerequisite for a usable count.

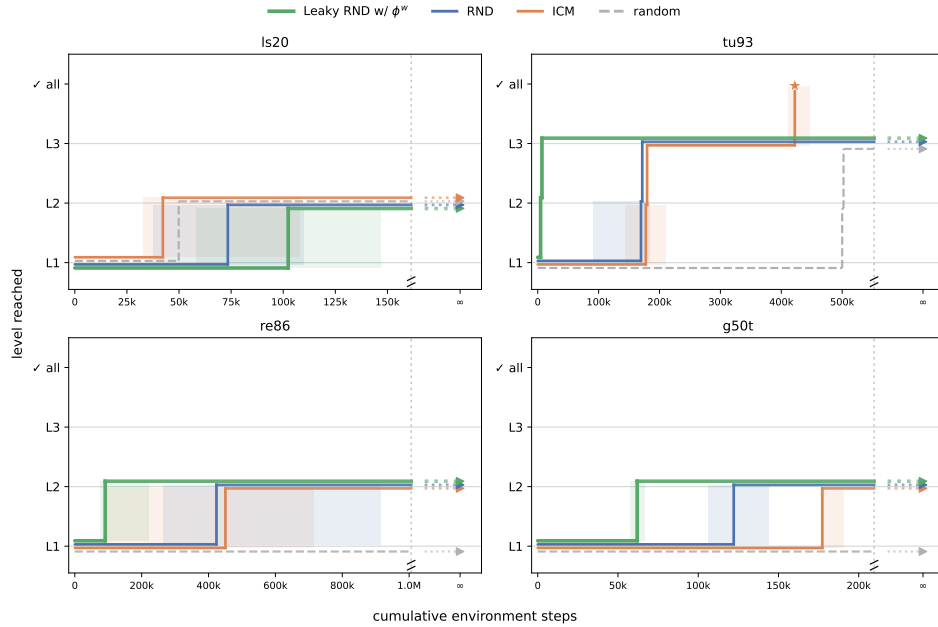


Figure 4: Cumulative environment steps each method requires to reach a given level, across four ARC-AGI-3 environments.

4.3 More Efficient Exploration with Leaky RND

We evaluated our method on four environments from ARC-AGI-3’s public dataset [6] (Appendix B), reporting the average number of environment steps to complete each level against RND (blue) [5] and ICM (orange) [12]. To isolate the effect of the intrinsic reward, we fixed the agent’s architecture and vary only the harness that generates the reward.

Across the environments, we see $\sim 35\%$ average reduction of steps compared to the best performing baseline. However, Leaky RND is unable to solve any of the levels previous methods couldn’t solve. The main challenge is that solving these levels requires exploiting the information collected through exploration, which is missing from existing methods (further discussed in Section 5.1).

4.4 Ablation Study

Table 1: **Component ablation** on four ARC-AGI-3 games (level one). Cells are the median environment steps to first reward over four seeds (lower is better).

Variant	ls20	tu93	re86	g50t
Full (Leaky RND on ϕ^w)	32k	132k	70k	56k
– leak (vanilla RND)	92k	292k	269k	28k
– ϕ^w (random ϕ)	43k	70k	474k	68k
– ϕ^w (pixel ϕ)	61k	149k	356k	98k
– harness (random policy)	50k	500k	1M+	1M+

We ablated the two main components of our method: the leak mechanism and the state representation ϕ^w (Table 1). Without the leak ($\mu = 0$), the agent uses roughly $2\text{--}4\times$ more steps to solve a level on ls20, tu93, and re86, confirming that continuously regenerating curiosity accelerates exploration. The exception is g50t, where eliminating the leak improves performance, presumably because its large number of distinct states makes forgetting which states have been visited harmful for exploration.

Switching ϕ^w to a randomly initialized encoder or a raw-pixel encoder consistently degrades performance, with tu93 as the lone exception. Finally, ablating the harness entirely removes the intrinsic reward, leaving the agent no learning signal and reducing it to random exploration. This baseline never solves re86 or g50t within 1M steps and is slower than Leaky RND on the two levels it does clear, confirming that the intrinsic reward, rather than chance, drives exploration.

5 Discussion

5.1 What is needed for ARC-AGI-3

Reflecting on our results, we believe that achieving progress on ARC-AGI-3 necessitates two things: general knowledge and intentionality.

General knowledge. Humans build their understanding of game mechanisms on top of concepts (e.g., clicking on a grid will attract nearby blobs toward it). Concepts carry associations and do the heavy weight-lifting during generalization. An agent solely trained on ARC-AGI-3 will not be exposed to this wide variety of concepts they can leverage to describe and model the world. They have to build their abstractions from scratch, and if they are in the form of opaque world models, then they further miss out the opportunity to utilize the priors human built.

Intentionality. Existing methods explore blindly. Humans, by contrast, explore with intent: we use what we observe to infer how a game works, then direct our actions accordingly (e.g., move to the bottom-left to test what the yellow blob does). This matters because completing a level often demands a specific action sequence (e.g., reach the cross, refill energy, reach the exit). Curiosity-based exploration has no mechanism for composing such structure. More fundamentally, it never exploits the information it gathers, lacking the reasoning capability to do so.

Hence, we believe general agents will emerge from LLMs, inheriting from language the priors needed for exploration, planning, and understanding.

5.2 Count-based exploration as implicit MCTS

Monte Carlo Tree Search (MCTS) works by using rollouts to iteratively refine value estimates of states. During rollouts, exploration and exploitation is balanced with UCT [8]. We realized that exploration with count-based bonuses can be interpreted as an amortized version of MCTS.

During action selection, the agent is rewarded for both exploitation (extrinsic reward) and exploration (count-based bonus), where the balance is implicitly represented in the policy. Alternatively, one can explicitly compute the UCT score and use that to guide action selection, armed with visit-counters like RND and other methods. One episode can be treated as a single rollout, where the terminal reward (leaf node value) is propagated upward through visited states via value function updates. Thus, count-bonus exploration amounts to a degenerate MCTS that searches a single tree with no explicit lookahead — explaining both its sample efficiency and its inability to plan toward distant goals.

6 Conclusion

In this work, we introduced Leaky RND, a count-based exploration method that prevents curiosity saturation by periodically pulling the distillator’s parameters toward their random initialization, leaking information so that curiosity is restored for states not visited recently. We demonstrated that a representation space learned through inverse-dynamics modeling is better suited to count-based exploration than a random feature space, particularly when distinct states are visually similar. Our method completes a level in roughly 35% fewer steps than the best baseline. However, it cannot solve any level that existing methods cannot. We argue that this reflects a fundamental limitation of curiosity-based exploration: it lacks the intentionality needed to direct exploration and act on the information it gathers. We believe a general agent capable of progressing on ARC-AGI-3 will need to emerge from LLMs, drawing on the vast priors and abstractions that humans have encoded in language.

References

- [1] Analyzing GPT-5.5 & opus 4.7 with ARC-AGI-3. URL <https://arcprize.org/blog/arc-agi-3-gpt-5-5-opus-4-7-analysis>.
- [2] Joshua Achiam and Shankar Sastry. Surprise-based intrinsic motivation for deep reinforcement learning. URL <http://arxiv.org/abs/1703.01732>.
- [3] Marc Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc. URL <https://proceedings.neurips.cc/paper/2016/hash/afda332245e2af431fb7b672a68b659d-Abstract.html>.
- [4] Yuri Burda, Harri Edwards, Deepak Pathak, Amos Storkey, Trevor Darrell, and Alexei A. Efros. Large-scale study of curiosity-driven learning, . URL <http://arxiv.org/abs/1808.04355>.
- [5] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation, . URL <http://arxiv.org/abs/1810.12894>.
- [6] ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. URL <http://arxiv.org/abs/2603.24621>.
- [7] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Gabriel Dulac-Arnold, Ian Osband, John Agapiou, Joel Z. Leibo, and Audrunas Gruslys. Deep q-learning from demonstrations. URL <http://arxiv.org/abs/1704.03732>.
- [8] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou, editors, *Machine Learning: ECML 2006*, volume 4212, pages 282–293. Springer Berlin Heidelberg. ISBN 978-3-540-45375-8 978-3-540-46056-5. doi: 10.1007/11871842_29. URL http://link.springer.com/10.1007/11871842_29. Series Title: Lecture Notes in Computer Science.

- [9] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc. URL https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.
- [10] Jean-Arcady Meyer and Stewart W. Wilson. A possibility for implementing curiosity and boredom in model-building neural controllers. In *From Animals to Animats: Proceedings of the First International Conference on Simulation of Adaptive Behavior*, pages 222–227. MIT Press. ISBN 978-0-262-25667-4. URL <https://ieeexplore.ieee.org/document/6294131>.
- [11] Georg Ostrovski, Marc G. Bellemare, Aäron Oord, and Rémi Munos. Count-based exploration with neural density models. In *International conference on machine learning*, pages 2721–2730. PMLR. URL <http://proceedings.mlr.press/v70/ostrovski17a.html>.
- [12] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction, . URL <http://arxiv.org/abs/1705.05363>.
- [13] Deepak Pathak, Dhiraj Gandhi, and Abhinav Gupta. Self-supervised exploration via disagreement, . URL <http://arxiv.org/abs/1906.04161>.
- [14] Tim Salimans and Richard Chen. Learning montezuma’s revenge from a single demonstration. URL <http://arxiv.org/abs/1812.03381>.
- [15] Jürgen Schmidhuber. Formal theory of creativity, fun, and intrinsic motivation (1990–2010). 2(3):230–247. ISSN 1943-0604, 1943-0612. doi: 10.1109/TAMD.2010.2056368. URL <http://ieeexplore.ieee.org/document/5508364/>.
- [16] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. URL <http://arxiv.org/abs/1707.06347>.
- [17] Mingqi Yuan. Intrinsically-motivated reinforcement learning: A brief introduction. URL <http://arxiv.org/abs/2203.02298>.

A Training Setup

A.1 Observation and Action Space

Each observation is a 64×64 grid of ARC palette indices over $n_{\text{colors}} = 16$ colors. We one-hot encode each grid into a $16 \times 64 \times 64$ tensor and apply no resizing, frame stacking, or cropping. The action space is the discrete ARC-AGI-3 action set; the per-environment action sets are listed in Appendix B. Each run trains on a single level; episodes start from that level’s initial state and end on completion or at a 200-step horizon, whichever comes first. The environment emits an extrinsic reward of +1 only on completion. This signal is never given to the agent; it is used solely to mark when a level is solved for the steps-to-solve metric (Appendix A.6).

A.2 Network Architectures

The agent encoder ϕ^π and the world encoder ϕ^w are two independent instances of the same convolutional architecture (Table 2); they share neither weights nor gradients. The inverse-dynamics (IDM) head, the RND target f_ψ , and the distillator $\hat{f}_\theta$ all operate on the 256-dimensional world representation h^w produced by ϕ^w (Table 3). All layers use orthogonal initialization — gain $\sqrt{2}$, except the policy head (0.01) and the value head (1.0) — with ReLU activations and no normalization or pooling.

A.3 World Encoder ϕ^w

We train ϕ^w solely with the inverse-dynamics objective of Equation (1), propagating no PPO gradient into it, using a dedicated Adam optimizer (Table 4). Before the main loop, we warm-start ϕ^w on transitions from 20 random-policy episodes, so that novelty is measured in a stable representation from the outset rather than in that of a freshly initialized encoder. These warm-up steps count toward both the step budget and the steps-to-solve metric. During training, ϕ^w continues to update online alongside the agent; the resulting representational drift is small and acts as a natural source of leakage.

Table 2: Shared CNN encoder. Input $16 \times 64 \times 64$ one-hot frame; output a 256-d feature vector. Used independently for ϕ^π and ϕ^w .

Layer	Type	Out ch.	Kernel	Stride	Pad
1	Conv2d + ReLU	32	3	1	1
2	Conv2d + ReLU	64	3	2	1
3	Conv2d + ReLU	64	3	2	1
4	Conv2d + ReLU	64	3	2	1
	Flatten	$64 \times 8 \times 8 = 4096$			
5	Linear + ReLU	$4096 \rightarrow 256$ (trunk)			

Table 3: Heads. Policy/value heads sit on ϕ^π ; the IDM head and Leaky RND networks sit on ϕ^w . $d = 256$ is the trunk/world-representation dim.

Module	Architecture	Notes
Policy head	Linear($d \rightarrow \mathcal{A} $)	gain 0.01
Value head	Linear($d \rightarrow 1$)	gain 1.0
IDM head	Linear($2d \rightarrow 256$), ReLU, Linear($256 \rightarrow \mathcal{A} $)	predicts $\hat{a}_t$
RND target f_ψ	Linear($d \rightarrow 256$), ReLU, Linear($256 \rightarrow 256$)	frozen at init
RND distillator $\hat{f}_\theta$	Linear($d \rightarrow 256$), ReLU, Linear($256 \rightarrow 256$), ReLU, Linear($256 \rightarrow 256$)	trainable (deeper than target)

A.4 Leaky RND

Leaky RND scores the raw novelty of a state as $\frac{1}{2} \text{mean}_j (\hat{f}_\theta(h^w)_j - f_\psi(h^w)_j)^2$, the mean squared distillation error between the distillator $\hat{f}_\theta$ and the frozen target f_ψ , and updates $\hat{f}_\theta$ toward f_ψ by minimizing $\mathcal{L}_{\text{Distill}}$ (Equation 2). After each distillation minibatch, we apply the leak update of Equation (3) with $\mu = 0.05$, pulling $\hat{f}_\theta$ back toward its random initialization. Transitions whose successor is a reset state are excluded from both the novelty bonus and the distillation update. Table 5 lists all hyperparameters.

Reward normalization. We map raw novelty to the intrinsic reward using running statistics, with three stabilizing measures. (i) The agent receives no intrinsic reward during the first 2 PPO updates, letting the distillator reach a stable error scale before its output drives the policy. (ii) We divide raw novelty by an exponential moving average (EMA, decay 0.99) of the standard deviation of the discounted intrinsic returns, applying no mean-centering so the bonus stays non-negative; we use an EMA rather than a cumulative estimate because the leak renders the novelty signal non-stationary. (iii) We clip per-step raw novelty at $5 \times$ its running mean before normalization, preventing transient spikes from corrupting the return statistics.

A.5 Agent and PPO

The agent is the convolutional actor-critic of Table 2, with separate policy and value heads, trained by PPO on the normalized intrinsic reward alone. Because each run stops at the first extrinsic reward, we use a single value head for the intrinsic return and estimate advantages with non-episodic generalized advantage estimation (GAE), bootstrapping across episode resets as in RND [5]. Table 6 lists all hyperparameters.

A.6 Training and Evaluation Protocol

Training on each level terminates at the first extrinsic reward. The reported metric is the cumulative number of environment steps up to that point, summed across the 16 parallel actors and inclusive of the warm-up steps. Runs that do not solve a level within its step budget are right-censored at the cap.

Table 4: World-encoder training hyperparameters.

Hyperparameter	Value
Optimizer	Adam, lr 1×10^{-3}
World representation dim $ h^w $	256
IDM hidden width	256
Warm-start episodes (random policy)	20
Warm-start epochs / batch size	4 / 256
Online update	one pass per PPO update, gradient clip 0.5

Table 5: Leaky RND hyperparameters.

Hyperparameter	Value
Target / distillator output dim	256
Distillator optimizer	Adam, lr $\eta = 1 \times 10^{-4}$
Distillation passes per update	1 (over rollout, 4 minibatches)
Leak rate μ	0.05
Leak cadence	after each distillation minibatch step
Normalization warm-up (updates)	2
Intrinsic-return EMA decay	0.99
Raw-novelty clip	$5 \times$ running mean

Each trainable module uses its own Adam optimizer — the agent, the world encoder ϕ^w together with its IDM head, and the distillator $\hat{f}_\theta$ — while the target network f_ψ remains frozen at initialization and is never updated. Per-level step budgets and the corresponding random-policy reference are given in Table 7, and the evaluation environments are described in Appendix B.

Seeds and aggregation. Seed counts vary by experiment. The main steps-to-solve results (Figure 4) use up to 4 seeds ($\{0, 1, 2, 3\}$) per (method $\times$ environment $\times$ level) cell on level 1, with fewer on the harder upper levels; for each cell we report the median steps-to-first-reward over the seeds that solved, with a 25–75th percentile band. The representation-space comparison (Section 4.2, Figure 3b) uses 5 seeds ($\{0, 1, 2, 3, 4\}$). The single-state curiosity-regeneration and counting probes (Figures 3a and 3b) use a single seed (0) and a single monitored state, since they isolate one state’s distillation error rather than aggregate performance. The ablation study (Section 4.4) uses 3 seeds ($\{0, 1, 2\}$).

The budgets above are for level 1; the main grid (Figure 4) also covers levels 2–3 under comparable per-cell caps (e.g., 1s20 level 2 at 300,000 steps). A uniform-random policy clears tu93 level 2 in $\sim 2,200$ steps but clears no other level shown.

Algorithm 1 summarizes the complete per-level training loop.

Algorithm 1 Leaky RND training loop (one level)

```

1: Warm-start  $\phi^w$  on 20 random-policy episodes (inverse dynamics, Eq. 1)
2: for each PPO update do
3:   Collect a rollout of  $T=128$  steps in  $N=16$  envs with policy  $\pi$ 
4:   Encode successors  $h_{t+1}^w = \phi^w(s_{t+1})$ 
5:   Raw novelty  $\leftarrow \frac{1}{2} \text{mean}_j (\hat{f}_\theta(h_{t+1}^w) - f_\psi(h_{t+1}^w))^2$ ; zero reset steps
6:    $r_t \leftarrow \text{normalize}(\text{raw novelty})$   $\triangleright$  warm-up, EMA-std, clip; Sec. A.4
7:   Estimate non-episodic GAE; update  $\pi$  and value head (PPO)
8:   Update  $\phi^w$  by inverse dynamics (Eq. 1)  $\triangleright$  online
9:   for each distillation minibatch do
10:    Step  $\hat{f}_\theta$  on  $\mathcal{L}_{\text{Distill}}$  (Eq. 2)
11:    Leak:  $\theta \leftarrow (1 - \mu)\theta + \mu\theta_{\text{init}}$   $\triangleright \mu = 0.05$ , Eq. 3
12:   end for
13:   if an extrinsic reward was observed then record step and stop
14: end for

```

Table 6: Agent / PPO hyperparameters.

Hyperparameter	Value
Parallel environments N	16
Rollout length T	128 (2048 transitions / update)
Optimizer	Adam, lr 3×10^{-4}
Discount γ (intrinsic)	0.99
GAE λ	0.95
PPO clip ϵ	0.2
Value-clip ϵ	0.2
Epochs / minibatches per update	4 / 4
Value loss coefficient	0.5
Entropy coefficient	0.01
Gradient clip (max norm)	0.5
Advantage normalization	per-minibatch

Table 7: Per-environment level-1 step budget (censoring cap) and the approximate expected steps-to-first-reward of a uniform-random policy on level 1.

Environment	$ \mathcal{A} $	Level-1 step budget	Random-policy E[steps] (L1)
1s20	4	200,000	$\sim 49,800$
tu93	4	600,000	$\sim 500,000$
re86	5	1,000,000	∞ (never clears)
g50t	5	300,000	∞ (never clears)

B Evaluation Environments

We evaluate on four environments drawn from ARC-AGI-3’s public set: 1s20, tu93, re86, and g50t. ARC-AGI-3 games are turn-based 64×64 grid puzzles that are presented without instructions: the objects, rules, and win condition of each level must be inferred purely through interaction, and each game is organized into successive levels that introduce new mechanics. We use the games unmodified and expose the agent to the intrinsic reward only.